

Day-3-Text-Processing and package management

2. Text Processing Tools

```
John,25,Engineer
Jane,30,Manager
Bob,22,Developer
Alice,28,Designer
Bob,25,Developer
Bob,24,Developer
Bob,26,Developer
```

```
sort data
```

grep - Search Text

```
# Basic search
grep "pattern" filename.txt

# Ingore Case insensitive
grep -i "pattern" filename.txt

# Show line numbers
grep -n "pattern"filename.txt

# Show context (lines before and after)
grep -C 3 "pattern" filename.txt
grep -B 2 "pattern" filename.txt      # Before
grep -A 2 "pattern" filename.txt      # After

# Invert match (show lines that DON'T contain pattern)
grep -v "pattern" filename.txt

# Search in multiple files
grep "pattern" *.txt

# match only whole words
grep -w "pattern" filename.txt

# Recursive search
grep -r "pattern" directory/

##
grep -r "hossam" /var/log/
# Regular expressions
grep "^start" filename.txt           # Lines starting with "start"
grep "end$" filename.txt             # Lines ending with "end"
grep "[0-9]" filename.txt            # Lines containing numbers
grep "A-Z" filename.txt.             # Finds lines with at least one uppercase letter.
```

sed - Stream Editor

```
# Substitute text
sed 's/old/new/' filename.txt
sed 's/old/new/g' filename.txt      # Global replacement

# Delete lines
sed '5d' filename.txt               # Delete line 5 or 3
sed -i '/man/d' filename.txt        # Delete lines containing pattern

# Print specific lines
sed -n '10,20p' filename.txt        # Print lines 10-20

# In-place editing
sed -i 's/old/new/g' filename.txt

# Multiple operations
sed -e 's/old1/new1/' -e 's/old2/new2/' filename.txt
```

awk - Text Processing

```
# Print specific columns
awk '{print $1, $3}' filename.txt    # Print 1st and 3rd columns

# Print lines with conditions
awk '$3 > 100' filename.txt          # Print lines where 3rd column > 100
awk -F ',' '$2 > 24' sorteddata

# Print with formatting
awk '{printf "Name: %s, Age: %s\n", $1, $2}' filename.txt

# Built-in variables
awk '{print NR, NF, $0}' filename.txt # NR=line number, NF=field count

# Field separators
awk -F ',' '{print $1}' filename.txt  # Use comma as separator

cat /etc/passwd | awk -F ':' '{print $1}'

for i in $(cat users) ; do echo $i; done
```

3. Pipes and Redirection

Pipes (|)

Pipes connect the output of one command to the input of another:

```
# Chain commands
ls -la | grep "txt"
cat filename.txt | grep "error" | wc -l
ps aux | grep "chrome" | awk '{print $2}'

# Sort and filter
cat /etc/passwd | cut -d: -f1 | sort | tail -10
```

Redirection

```
# Output redirection
echo "Hello" > output.txt          # Overwrite file
echo "World" >> output.txt          # Append to file

# Input redirection
sort < input.txt > output.txt

# Error redirection
command 2> error.log               # Redirect errors
command > output.log 2>&1          # Redirect both output and errors
./script > data 2> errorr
command &> all.log                  # Redirect both (shorthand)

# Discard output
command > /dev/null                # Discard standard output
command 2> /dev/null               # Discard errors
command &> /dev/null                # Discard everything
```

4. Finding Files and Content

find - Locate Files

```
# Find by name
find /home -name "*.txt"
find . -name "config*"
find / -name "filename" 2>/dev/null # Suppress permission errors

# Find by type
find . -type f            # Files only
find . -type d            # Directories only
```

```
# Find by size
find . -size +100M          # Files larger than 100MB
find . -size -1k           # Files smaller than 1KB

# Find by modification time
find . -mtime -7           # Modified in last 7 days
find . -mtime +30          # Modified more than 30 days ago

# Execute commands on found files
find . -name "*.tmp" -delete
find . -name "*.log" -exec rm {} \;

sudo find /var/log/ -name "*.log" -exec cat {} \; > systemlogs
```

which and whereis

```
# Find executable location
which python
which ls

# Find program, source, and manual
whereis python
whereis ls
```

5. Text Editors

nano - Simple Editor

```
# Open file
nano filename.txt

# Nano shortcuts
Ctrl + O      # Save (Write Out)
Ctrl + X      # Exit
Ctrl + K      # Cut line
Ctrl + U      # Paste
Ctrl + W      # Search
Ctrl + G      # Help
```

vim - Advanced Editor

```
# Open file
vim filename.txt

# Vim modes
# Normal mode (default) - for navigation and commands
# Insert mode - for typing text
# Command mode - for saving, quitting, etc.

# Basic vim commands
i          # Enter insert mode
Esc        # Return to normal mode
:w         # Save
:q         # Quit
:wq        # Save and quit
:q!        # Quit without saving
dd         # Delete line
yy         # Copy line
p          # Paste
/pattern:  # Search
n          # Next search result
```

6. File Links and Archives

Symbolic Links

```
# Create symbolic link
ln -s /path/to/original /path/to/link
```

```
# Example
ln -s /usr/bin/python3 /usr/local/bin/python

# List links
ls -la | grep "^l"

# Remove link
rm link_name
```

Archives and Compression

```
# Create tar archive
tar -cvf archive.tar files/
tar -czvf archive.tar.gz files/      # Compressed with gzip
tar -cjvf archive.tar.bz2 files/     # Compressed with bzip2

# Extract tar archive
tar -xvf archive.tar
tar -xzvf archive.tar.gz
tar -xjvf archive.tar.bz2

# List archive contents
tar -tvf archive.tar

# Compress files
only works on individual files
gzip filename.txt                  # Creates filename.txt.gz
gunzip filename.txt.gz             # Decompresses

# Zip files
zip archive.zip files/
unzip archive.zip
```

7. Hands-on Practice

Exercise 1: Permissions

1. Create a file called `secret.txt` with some content
2. Check its permissions with `ls -l`
3. Change permissions to 600 (owner read/write only)
4. Try to read it as another user
5. Change permissions back to 644

Exercise 2: Text Processing

1. Create a file with the following content:

```
John,25,Engineer
Jane,30,Manager
Bob,22,Developer
Alice,28,Designer
```

2. Use `grep` to find lines containing "Engineer"
3. Use `awk` to print only names and ages
4. Use `sed` to change "Developer" to "Programmer"

```
# Print names and ages only
awk -F',' '{print $1, $2}' employees.csv

# Find people older than 28
awk -F',' '$2 > 28' employees.csv

# Calculate average age
awk -F',' '{sum += $2} END {print "Average age:", sum/NR}' employees.csv

# Count people by position
awk -F',' '{count[$3]++} END {for (i in count) print i, count[i]}' employees.csv

# Add line numbers and field count
awk -F',' '{print NR, NF, $0}' employees.csv
```

Exercise 3: Pipes and Redirection

- List all files in your home directory
- Save the list to a file called `filelist.txt`
- Count how many files you have
- Find the largest file in your home directory

Exercise 4: File Operations

- Create a directory called `backup`
- Copy some files to the backup directory
- Create a tar archive of the backup directory
- Remove the original backup directory
- Extract the archive to verify it works

8. Key Takeaways

Essential Commands Learned Today

- `chmod` , `chown` , `chgrp` - Permission management
- `grep` , `sed` , `awk` - Text processing
- `|` , `>` , `>>` , `<` - Pipes and redirection
- `find` , `which` - File searching
- `nano` , `vim` - Text editing
- `tar` , `gzip` , `zip` - Archives and compression

Best Practices

- Use `grep -r` for recursive searches
- Always test `sed` and `awk` commands before using `-i` flag
- Use `find` with `2>/dev/null` to suppress permission errors
- Learn vim gradually - start with basic commands
- Always verify archives before deleting originals

9. Homework

- Create a script that:
 - Finds all `.log` files in `/var/log`
 - Searches for "error" in each file
 - Saves the results to `error_report.txt`
- Practice vim by creating and editing a file
- Create a backup of your important files using tar
- Experiment with different permission combinations